

MSC QUANTUM SCIENCE AND TECHNOLOGY

MASTER THESIS

---

# **Toward ML-Based State Discrimination on FPGA in Open-Source Quantum Control Electronics**

---

**Author: Ender Jose Barillas Rodriguez**

**Supervised by: Joel Pérez Díaz**

NIUB: 23254663  
Faculty of Physics, University of Barcelona  
08028, Barcelona  
July 2026

## Acknowledgements

I would like to thank the team at Qilimanjaro Quantum Tech for their guidance and support throughout this project. Special thanks to Joel Pérez Díaz for supervising this work and to all colleagues who offered technical help and stimulating discussions. This thesis was carried out during an internship at Qilimanjaro from February to July 2026. **[TODO: Expand. Thank David A. for reviewing, and Fabio H. and the team for help with the board. Thank classmates. Thank Bruno and the program for letting me take a shot at joining a new field. Thank Arianna for helping me survive in Barcelona. Thank parents as well.]**

## Abstract

Low-latency and high-fidelity qubit state measurement is essential for superconducting quantum computing, particularly for enabling mid-circuit measurements and real-time feedback. Conventional readout techniques rely on host-side processing, introducing delays that limit measurements to the end of a quantum circuit. FPGA-based approaches offer a path toward in-situ state discrimination, but many existing solutions are specialised or tightly coupled to specific hardware designs.

This thesis presents a general-purpose readout architecture based on the QICK framework [1], implemented on a Xilinx ZCU216 RFSoc platform, that supports both mid-circuit and end-of-circuit quantum state measurements. The work encompasses full system integration, including FPGA configuration and remote operation, QICK-based calibration, and the design of a real-time readout data path. Control and readout waveforms are generated using Direct Digital Synthesis (DDS), and the role of the analogue front end is examined to ensure reliable signal transmission and measurement.

Building on this infrastructure, we investigate machine-learning-assisted state discrimination within QICK to accelerate readout while maintaining accuracy. Lightweight machine-learning methods suitable for FPGA deployment are evaluated with respect to latency and computational cost. The resulting architecture enables low-latency state classification directly on the control hardware, reducing reliance on host-side processing and providing a unified measurement pipeline for dynamic quantum circuits.

*Keywords:* superconducting qubits, readout, FPGA, QICK, RFSoc, dispersive measurement, mid-circuit measurement, machine learning

# Contents

|          |                                                       |           |
|----------|-------------------------------------------------------|-----------|
| <b>1</b> | <b>Introduction</b>                                   | <b>1</b>  |
| 1.1      | Motivation . . . . .                                  | 1         |
| 1.2      | Goals and Contributions . . . . .                     | 1         |
| 1.3      | Thesis Outline . . . . .                              | 2         |
| <b>2</b> | <b>Theoretical Background</b>                         | <b>2</b>  |
| 2.1      | Superconducting Qubits . . . . .                      | 2         |
| 2.2      | Circuit Quantum Electrodynamics . . . . .             | 2         |
| 2.3      | Dispersive Readout . . . . .                          | 3         |
| 2.4      | Direct Digital Synthesis . . . . .                    | 3         |
| 2.5      | State Discrimination . . . . .                        | 3         |
| <b>3</b> | <b>Hardware Platform</b>                              | <b>4</b>  |
| 3.1      | Xilinx ZCU216 RFSoc . . . . .                         | 4         |
| 3.2      | QICK Software and Firmware . . . . .                  | 4         |
| 3.3      | XM655 Analogue Front-End Module . . . . .             | 5         |
| <b>4</b> | <b>System Integration</b>                             | <b>5</b>  |
| 4.1      | FPGA Configuration and Board Bring-up . . . . .       | 5         |
| 4.2      | Channel Mapping . . . . .                             | 6         |
| 4.3      | Phase Calibration . . . . .                           | 6         |
| <b>5</b> | <b>Readout Architecture</b>                           | <b>7</b>  |
| 5.1      | DDS-Based Signal Generation . . . . .                 | 7         |
| 5.2      | Decimated vs. Accumulated Readout . . . . .           | 8         |
| 5.3      | Resonator Spectroscopy . . . . .                      | 8         |
| 5.4      | Multiplexed Readout . . . . .                         | 9         |
| <b>6</b> | <b>Real-Time Feedback and Mid-Circuit Measurement</b> | <b>9</b>  |
| 6.1      | tProcessor Architecture . . . . .                     | 9         |
| 6.2      | Conditional Logic Demonstration . . . . .             | 10        |
| 6.3      | $\pi$ Pulse Calibration . . . . .                     | 10        |
| 6.4      | Active Qubit Reset . . . . .                          | 11        |
| 6.5      | Latency Budget . . . . .                              | 11        |
| <b>7</b> | <b>ML-Assisted State Discrimination</b>               | <b>12</b> |
| 7.1      | IQ Data and the Discrimination Problem . . . . .      | 12        |
| 7.2      | Lightweight Classifiers for FPGA Deployment . . . . . | 12        |
| 7.3      | Training and Evaluation Pipeline . . . . .            | 13        |
| 7.4      | Fidelity Metrics . . . . .                            | 13        |
| <b>8</b> | <b>Conclusions and Outlook</b>                        | <b>13</b> |
| 8.1      | Summary . . . . .                                     | 13        |
| 8.2      | Outlook . . . . .                                     | 14        |
|          | <b>Bibliography</b>                                   | <b>15</b> |
| <b>A</b> | <b>Key Code Listings</b>                              | <b>16</b> |
| A.1      | Phase Calibration Programme . . . . .                 | 16        |

|     |                                                                   |    |
|-----|-------------------------------------------------------------------|----|
| A.2 | Fast Amplitude Sweep with <code>RAveragerProgram</code> . . . . . | 16 |
| A.3 | Active Reset Sequence (Pseudocode) . . . . .                      | 17 |

# 1 Introduction

Quantum computing holds the promise of solving classically intractable problems in optimisation, simulation, and cryptography. Among the leading hardware platforms, superconducting qubits stand out for their relatively fast gate times, high coherence, and compatibility with existing microwave engineering infrastructure [2]. A central challenge that must be overcome on the road to fault-tolerant operation is *qubit state measurement*: reading out the quantum information stored in a qubit quickly, accurately, and with minimal back-action on unmeasured qubits.

## 1.1 Motivation

Classical error correction requires measurements that are not only high-fidelity but also *fast* relative to qubit coherence times, and — crucially for quantum error correction — it requires *mid-circuit measurements*: the ability to measure ancilla qubits in the middle of a quantum algorithm, use the result to apply corrections in real time, and continue the computation. This capability demands a control electronics stack capable of making a state discrimination decision on-chip in nanoseconds to microseconds, not the milliseconds typical of host-software processing.

Field-Programmable Gate Arrays (FPGAs) offer a compelling solution. Their deterministic, sub-microsecond timing enables real-time signal processing close to the qubit hardware. The Quantum Instrumentation Control Kit (QICK) [1] is an open-source FPGA firmware and software framework that turns Xilinx Radio-Frequency System-on-Chip (RF-SoC) boards into complete qubit controllers, combining fast digital-to-analogue converters (DACs), analogue-to-digital converters (ADCs), and a tightly integrated soft-processor into a single compact platform.

## 1.2 Goals and Contributions

Despite the availability of QICK, assembling a *general-purpose* readout pipeline from an uncharacterised board to a working single-shot state discriminator requires integrating many layers: physical wiring, FPGA configuration, phase calibration, signal optimisation, and machine-learning post-processing. The lack of a unified reference implementation makes this process time-consuming and error-prone, especially for multi-qubit scenarios.

This thesis makes the following contributions:

1. **Full-stack integration** of the QICK framework on a Xilinx ZCU216 RFSoc board with an XM655 analogue front-end module, including remote operation via a persistent Pyro4 server.
2. **Phase-coherent readout calibration**: a systematic procedure for measuring and compensating the random DAC–ADC phase offset that arises at every board power-on, enabling reproducible IQ measurements across sessions.
3. **Readout data-path design**: a characterisation of the trade-offs between decimated and accumulated readout modes and a complete resonator spectroscopy workflow suitable for both single-qubit and multiplexed measurements.
4. **Real-time feedback primitives**: implementation and demonstration of conditional pulse logic on the tProcessor, establishing the active qubit reset and mid-circuit measurement infrastructure needed for dynamic quantum circuits.

5. **ML-assisted state discrimination:** evaluation of lightweight machine-learning classifiers for on-FPGA deployment, balancing fidelity, latency, and resource consumption.

### 1.3 Thesis Outline

Section 2 reviews the relevant theory of superconducting qubits, circuit quantum electrodynamics (cQED), and dispersive readout. Section 3 describes the hardware platform. Section 4 covers system integration and calibration. Section 5 presents the readout architecture and measurement workflows. Section 6 describes real-time feedback and mid-circuit measurement. Section 7 evaluates machine-learning-based state discrimination. Section 8 summarises the results and outlines future directions.

## 2 Theoretical Background

### 2.1 Superconducting Qubits

Superconducting qubits are anharmonic LC oscillators whose nonlinearity is provided by one or more Josephson junctions [2]. The Josephson junction acts as a nonlinear inductor with energy  $E_J \cos \hat{\varphi}$ , where  $\hat{\varphi}$  is the superconducting phase across the junction and  $E_J$  is the Josephson energy. Combined with a shunt capacitance  $C$  (charging energy  $E_C = e^2/2C$ ), the system forms an artificial atom with a discrete, anharmonic spectrum that can be addressed at the transition frequency  $\omega_{01}$ .

**Transmon.** The transmon [3] operates deep in the regime  $E_J \gg E_C$ , making its energy levels insensitive to charge noise. Its transition frequency lies typically in the 4 GHz to 8 GHz range, and its anharmonicity  $\alpha = \omega_{12} - \omega_{01}$  is of order  $-E_C/\hbar$ , typically a few hundred MHz.

**Fluxonium.** The fluxonium [4] replaces the transmon's large capacitor with a superinductance  $L$  (inductive energy  $E_L = (\Phi_0/2\pi)^2/L$ , where  $\Phi_0$  is the flux quantum), threading the loop with an external flux  $\Phi_z$ . The resulting double-well potential can be tuned between a plasmon regime (near  $\Phi_z = 0$ ) and a fluxon regime (near  $\Phi_z = \Phi_0/2$ ). Fluxonium qubits offer long coherence times and are of particular interest because their readout landscape is richer: at half-integer flux bias the two wells become degenerate and the qubit transition frequency passes through zero, enabling so-called *Which-Well Readout* (WWR) [5].

### 2.2 Circuit Quantum Electrodynamics

In the circuit QED (cQED) architecture [6], a qubit is coupled dispersively to a microwave resonator. The resonator acts both as a readout bus and as a filter that protects the qubit from radiative decay into the measurement chain. The full Hamiltonian (setting  $\hbar = 1$ ) is

$$\hat{H} = \hat{H}_{\text{qubit}} + \omega_r \hat{a}^\dagger \hat{a} + g \hat{n}_q (\hat{a} + \hat{a}^\dagger), \quad (1)$$

where  $\omega_r$  is the resonator frequency,  $\hat{a}$  ( $\hat{a}^\dagger$ ) is the resonator annihilation (creation) operator,  $g$  is the qubit–resonator coupling strength, and  $\hat{n}_q$  is the relevant qubit operator (charge for capacitive coupling).

### 2.3 Dispersive Readout

When  $|g_{ij}| \ll |\omega_{ij} - \omega_r|$  for all relevant qubit transitions  $i \leftrightarrow j$ , second-order perturbation theory in  $g$  yields a qubit-state-dependent shift of the resonator frequency [5]:

$$\hat{H}_{\text{disp}} = \left( \omega_r + \sum_i \chi_i |i\rangle \langle i| \right) \hat{a}^\dagger \hat{a}, \quad \chi_i = \sum_{j \neq i} \frac{|g_{ij}|^2}{\omega_{ij} - \omega_r} - \frac{|g_{ji}|^2}{\omega_{ji} - \omega_r}, \quad (2)$$

where  $g_{ij} = \langle i | g \hat{n}_q | j \rangle$  and  $\omega_{ij} = \omega_j - \omega_i$ . The textbook two-level dispersive shift  $\chi = \chi_1 - \chi_0$  follows from truncating to the computational subspace, but for fluxonium-class qubits the straddling regime (where  $|g_{12}| \gg |g_{01}|$ ) makes this truncation unsafe; higher levels must be included [3].

In practice, one probes the resonator at its bare frequency  $\omega_r$  and observes a state-dependent phase shift of the transmitted or reflected signal. For a single-qubit system, the resonator sits at  $\omega_r + \chi_0$  when the qubit is in  $|0\rangle$  and at  $\omega_r + \chi_1$  when in  $|1\rangle$ , resulting in an IQ-plane separation that grows with the dimensionless cavity pull  $2|\chi|/\kappa$ , where  $\kappa$  is the resonator linewidth [6].

**Which-Well Readout.** For a double-well fluxonium biased off degeneracy, the eigenstates are approximately localised in the left ( $|L\rangle$ ) or right ( $|R\rangle$ ) well and carry definite persistent-current sign. Because the two wells host different local spectra, the dispersive shift evaluated on the ground state of each well differs, and the half-splitting  $\chi_{\text{WWR}} = \frac{1}{2}(\chi_R - \chi_L)$  sets the WWR contrast [5].

### 2.4 Direct Digital Synthesis

Direct Digital Synthesis (DDS) generates analogue waveforms by accumulating a phase register at a fixed clock rate and mapping the instantaneous phase value to a digital amplitude via a look-up table. A DDS numerically controlled oscillator (NCO) produces a tone at frequency

$$f_{\text{out}} = \frac{f_{\text{clk}} \cdot \Delta\phi}{2^N}, \quad (3)$$

where  $f_{\text{clk}}$  is the system clock,  $\Delta\phi$  is the phase increment per clock cycle, and  $N$  is the bit width of the phase accumulator. DDS avoids the spur and image problems associated with analogue IQ mixers because all imperfections are digital and predictable. In QICK, the tProcessor drives DDS engines on both the DAC (transmit) and ADC (receive) sides, enabling phase-locked signal generation and coherent demodulation [1].

### 2.5 State Discrimination

Following dispersive readout, the measured IQ quadrature amplitudes ( $I, Q$ ) form two clusters in the complex plane — one for  $|0\rangle$  and one for  $|1\rangle$ . Single-shot state discrimination assigns each measurement to one of these clusters. The simplest approach is a linear threshold: a hyperplane in the IQ plane that minimises the total misclassification probability, which for Gaussian clusters of equal covariance is the perpendicular bisector of the line connecting the centroids [7]. The single-shot readout fidelity is

$$\mathcal{F} = 1 - \frac{1}{2} (p(1|0) + p(0|1)), \quad (4)$$

where  $p(1|0)$  is the probability of assigning  $|1\rangle$  given that the qubit was prepared in  $|0\rangle$ , and vice versa. More sophisticated classifiers — quadratic discriminant analysis, neural



networks — can improve fidelity when the clusters are non-Gaussian or when multi-level leakage is present.

### 3 Hardware Platform

#### 3.1 Xilinx ZCU216 RFSoc

The Xilinx Zynq UltraScale+ ZCU216 evaluation board is built around an RFSoc device that integrates, on a single chip, a quad-core ARM Cortex-A53 processor, an FPGA fabric, and 16 RF-DAC and 16 RF-ADC channels with sampling rates up to 9.85 GS/s and 2.5 GS/s, respectively. Operating at 12-bit and 14-bit resolution, the converters can directly synthesise and capture signals in the 0 GHz to 6 GHz range without an external frequency up-conversion stage, provided the analogue signal chain is suitably conditioned.

The board runs the PYNQ Linux distribution, which exposes the FPGA fabric and the converters through Python libraries, making it straightforward to implement and iterate on firmware/software stacks. Table 1 summarises the key specifications relevant to this thesis.

Table 1: Selected ZCU216 specifications relevant to qubit readout.

| Parameter        | Value           | Note              |
|------------------|-----------------|-------------------|
| DAC channels     | 16              | Differential GSPS |
| DAC rate         | up to 9.85 GS/s | Per channel       |
| DAC resolution   | 14 bit          |                   |
| ADC channels     | 16              | Differential GSPS |
| ADC rate         | up to 2.5 GS/s  | Per channel       |
| ADC resolution   | 12 bit          |                   |
| FPGA fabric      | UltraScale+     | 930k LUT          |
| Processor        | 4× Cortex-A53   | 1.5 GHz           |
| Operating system | PYNQ Linux      |                   |

#### 3.2 QICK Software and Firmware

The Quantum Instrumentation Control Kit (QICK) [1] is an open-source framework originally developed at Fermilab that turns an RFSoc board into a self-contained qubit controller. It consists of two layers:

**Firmware.** A set of Vivado IP blocks synthesised and placed onto the FPGA fabric, comprising:

- **Signal generators:** DDS-based waveform engines that drive the DACs with arbitrary envelopes, constant tones, or flat-top pulses. Generator 6 (physical port 2\_231 on the ZCU216) is used as the primary readout generator throughout this work.
- **Readout blocks:** ADC-side DDS demodulators that down-convert the received signal to baseband. Each readout block provides both a decimated output (time-domain I/Q waveform) and an accumulation buffer (averaged I/Q pair). Readout channel 0 (ADC port 0\_226) corresponds to the first ADC pair on the XM655 breakout.

- **tProcessor-64**: a soft-processor that orchestrates pulse timing with nanosecond precision. It executes a custom instruction set including direct memory writes (**memri**), register arithmetic (**mathi**, **regwi**), conditional branches (**condj**), and pulse triggers. The ZCU216 uses the 64-bit variant of the tProcessor, which extends the original 32-bit design with wider arithmetic and richer branching.

**Software.** A Python library (the **qick** package) that runs on the ARM core and exposes high-level classes for writing and running experiments:

- **QickSoc**: instantiates the full hardware stack, loads the bitstream, and provides access to the converters and tProcessor.
- **QickProgram** / **AveragerProgram** / **RAveragerProgram**: abstractions for writing measurement sequences at increasing levels of automation. **RAveragerProgram** enables FPGA-internal parameter sweeps via the tProcessor’s **update()** hook, avoiding Python overhead between experiment points.
- **Remote operation**: a persistent Pyro4 server running on the board makes it possible to control the board from a laptop over a standard network connection, without reinitialising the FPGA between notebook sessions. This design also preserves the phase calibration across reconnects.

### 3.3 XM655 Analogue Front-End Module

The ZCU216’s RF converters use differential signalling on High-Density (HD) headers (JHC3 and JHC5 on the board). Most laboratory equipment, however, uses single-ended SMA connectors. The XM655 module is Xilinx’s daughter-card solution: it provides BALUN (balanced-to-unbalanced) transformers that convert the differential DAC outputs and ADC inputs to/from single-ended SMA ports, along with optional filtering.

Key constraints introduced by the XM655 in this work:

- The BALUNs on the XM655 cover the frequency range 10 MHz to 1000 MHz. Resonator frequencies above 1 GHz require connections directly on the HD header pins using pigtail cables and separate discrete BALUNs.
- Only a limited number of BALUN-equipped SMA ports are available in the 3 GHz to 6 GHz band, constraining the number of simultaneous readout tones.

The loopback path used for calibration experiments consists of: DAC port 2\_231 (JHC3) → HD2-SMA pigtail → BALUN → SMA cable → BALUN → HD2-SMA pigtail → ADC port 0\_226 (JHC5).

Figure 1 shows the high-level signal flow from the tProcessor to the qubit chip and back.

## 4 System Integration

### 4.1 FPGA Configuration and Board Bring-up

The ZCU216 boots the PYNQ Linux image from an SD card. After boot, the **QICK** bitstream is loaded onto the FPGA fabric by instantiating a **QickSoc** object in Python, which calls the PYNQ overlay loader and configures all IP blocks. Because the RFSoc

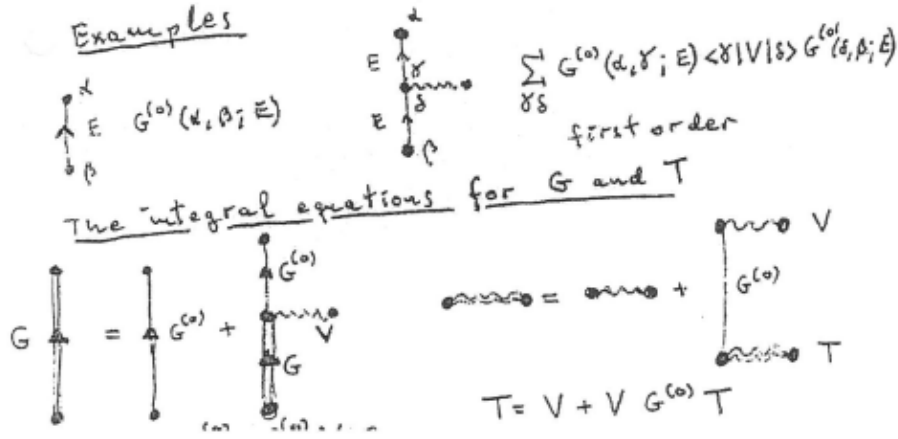


Figure 1: System topology placeholder. The tProcessor generates pulse sequences that drive the DAC signal generators; the analogue signal passes through the XM655 front-end to the qubit chip. The reflected/transmitted readout tone is routed through the cryogenic chain back to the ADC via the XM655. **[TODO: Replace with actual system diagram.]**

converters are controlled through AXI register maps rather than external chip-select lines, initialisation is entirely software-driven.

The board is accessed over SSH. To avoid re-initialising the hardware for every new notebook session, a persistent Pyro4 object server is launched once on the board and kept running:

```
1 # On the board (runs once per power cycle)
2 import Pyro4, qick
3 soc = qick.QickSoc()
4 daemon = Pyro4.Daemon()
5 uri = daemon.register(soc, "qick.soc")
6 Pyro4.locateNS().register("qick.soc", uri)
7 daemon.requestLoop()
```

From a remote laptop:

```
1 import Pyro4
2 soc = Pyro4.Proxy("PYRONAME:qick.soc")
3 soccfg = soc # lightweight config object
```

This design preserves the phase calibration state (see Section 4.3) across reconnects and makes experiment iteration fast.

## 4.2 Channel Mapping

The QICK firmware assigns each physical DAC output to a numbered *generator channel* and each physical ADC input to a numbered *readout channel*. On the ZCU216 with the default QICK bitstream, the mapping relevant to this work is shown in Table 2.

## 4.3 Phase Calibration

Each time the ZCU216 is powered on or the FPGA bitstream is reloaded, the DDS oscillators in the DAC and ADC blocks boot with a random mutual phase offset  $\phi$ . Without compensation, IQ measurements rotate unpredictably between sessions, making it impossible to compare results across power cycles.

Table 2: QICK channel mapping on the ZCU216.

| QICK Channel | Physical Port | HD Header | Function                |
|--------------|---------------|-----------|-------------------------|
| Generator 6  | DAC 2_231     | JHC3      | Readout / control drive |
| Readout 0    | ADC 0_226     | JHC5      | IQ acquisition          |

**Phase model.** A loopback tone from DAC 2\_231 to ADC 0\_226 measures  $\phi$  at a given frequency. Because the two oscillators are effectively separated by a fixed time delay  $\tau$  (the round-trip propagation delay in the loopback cable), the phase offset scales linearly with frequency:

$$\phi(f) = \phi_0 - 360^\circ \cdot \tau \cdot f, \quad (5)$$

where  $\phi_0$  is a constant offset and  $\tau \approx 251 \mu\text{s}$  in the as-assembled loopback (dominated by the cable length, not the on-chip delay).

**Calibration procedure.** The phase calibration sweeps a range of frequencies  $f_i$  near the target readout frequency, records the averaged IQ phasor at each point, and extracts  $\phi_i = \arg\langle I + jQ \rangle$ . The pair  $(\phi_0, \tau)$  is then fitted using `scipy.optimize.least_squares`, taking care to handle the  $0^\circ/360^\circ$  phase wrap correctly. The fit is considered reliable when the signal-to-noise ratio satisfies  $|\text{mag}| \gg \text{RMS}$  (e.g.  $637 \gg 1.8$  ADU in a typical calibration run).

The resulting `res_phase` value:

$$\phi_{\text{res}} = \phi_0 - 360^\circ \cdot \tau \cdot f_{\text{res}} \quad (6)$$

is stored in the experiment configuration dictionary and passed to the readout pulse via `soccfg.deg2reg(phi_res)`, which converts degrees to the integer register format expected by the tProcessor. This calibration must be re-run after every board reboot or bitstream reload, but remains valid throughout a measurement session.

**Measurement cadence.** A summary of the recommended start-of-session checklist is:

1. Power on and wait for PYNQ to boot ( $\approx 60$  s).
2. Connect the loopback (DAC 2\_231  $\rightarrow$  BALUN  $\rightarrow$  SMA  $\rightarrow$  BALUN  $\rightarrow$  ADC 0\_226) and run the phase calibration notebook.
3. Identify the correct `adc_trig_offset` using decimated readout (see Section 5.2).
4. Switch to accumulated readout for all data-taking.
5. At session end, call `soc.reset_gens()` to halt all running generators.

## 5 Readout Architecture

### 5.1 DDS-Based Signal Generation

In a standard heterodyne readout chain, a local oscillator (LO) at frequency  $f_{\text{LO}}$  is mixed with a baseband envelope to up-convert the readout tone to the resonator frequency  $f_r$ . This analogue mixing process introduces image products and LO leakage that degrade fidelity, requiring careful IQ imbalance correction.

QICK avoids analogue mixing entirely. Both the readout drive and the demodulation reference are produced by the on-chip DDS engines, which operate at the direct output frequency of the RFSoc converters (up to several GHz). The waveform generated for qubit readout has the form

$$s(t) = A \cdot \text{env}(t) \cdot \cos(2\pi f_r t + \phi_{\text{res}}), \quad (7)$$

where  $\text{env}(t)$  is the pulse envelope (constant, Gaussian, DRAG, etc.),  $f_r$  is programmed directly into the DDS frequency register, and  $\phi_{\text{res}}$  is the phase correction from calibration. The only noise sources are digital: quantisation, clock jitter, and DDS phase truncation spurs — all of which are deterministic and can be mitigated by appropriate filtering and sample-rate choice.

## 5.2 Decimated vs. Accumulated Readout

After the ADC samples the incoming signal, the QICK readout block demodulates it by multiplying with the NCO reference tone and low-pass filtering. Two output modes are available:

**Decimated readout.** Stores the full time-domain I/Q waveform over the readout window (e.g. 1000 samples). This is essential for setup: it reveals when the readout pulse arrives relative to the trigger (`adc_trig_offset`), whether the integration window is correctly positioned, and whether the signal shape is as expected. The single-shot noise is  $\sim 1.9$  ADU RMS.

**Accumulated readout.** The FPGA sums all ADC samples in the readout window into a single  $(I, Q)$  pair in hardware. With  $N_{\text{reps}}$  repetitions, the noise falls as  $1/\sqrt{N_{\text{reps}}}$ :

$$\sigma_{\text{accum}} = \frac{\sigma_{\text{single}}}{\sqrt{N_{\text{reps}}}} \approx \frac{1.9 \text{ ADU}}{\sqrt{N_{\text{reps}}}}. \quad (8)$$

At  $N_{\text{reps}} = 1000$  this yields  $\sigma \approx 0.14$  ADU — a  $31.6\times$  noise reduction. All experimental data in this thesis use accumulated readout once the timing and phase have been verified in decimated mode.

The practical workflow is therefore: (1) use `acquire_decimated()` to find the correct `adc_trig_offset` and verify signal quality; (2) switch to `acquire()` (accumulated) for all data-taking.

## 5.3 Resonator Spectroscopy

The resonator bare frequency  $\omega_r$  and linewidth  $\kappa$  are found by sweeping the probe frequency around the estimated resonator frequency and measuring the transmitted or reflected IQ amplitude. In QICK, there are two sweep strategies:

**Fast sweep (RAveragerProgram).** The tProcessor increments the DAC DDS frequency register at each experiment step via the `update()` method. This approach is fast because the FPGA handles all steps with no Python round-trips. However, the ADC NCO frequency is set via the ARM AXI bus and cannot be updated by the tProcessor. For resonator spectroscopy, both the DAC and ADC frequencies must track together; this is not achievable with the fast method.

**Slow sweep (Python outer loop).** A `for` loop in Python updates both the DAC generator frequency and the ADC readout frequency at each step. The per-step overhead is a few milliseconds (dominated by network round-trips in remote mode), but this approach can sweep *any* parameter, including the ADC NCO. This is the correct approach for resonator spectroscopy.

In practice:

```

1 freqs = np.linspace(f_start, f_stop, n_points)
2 amps = []
3 for f in freqs:
4     cfg["pulse_freq"] = f
5     prog = AveragerProgram(soccfg, cfg)
6     iq = prog.acquire(soc, load_pulses=True, progress=False)
7     amps.append(np.abs(iq[0][0][0] + 1j*iq[0][0][1]))

```

The resonator frequency is identified as the dip (reflection) or peak (transmission) in the swept amplitude response.

## 5.4 Multiplexed Readout

For multi-qubit systems, simultaneous readout of several qubits on the same physical line requires *multiplexed readout*: a broadband probe signal containing multiple tones at the individual resonator frequencies  $\{f_{r,k}\}$ . The QUICK framework supports this through a *Polyphase Filter Bank* (PFB) readout mode.

A PFB is a bank of digital bandpass filters implemented as a single, efficient DSP operation (polyphase decomposition of a prototype filter followed by an FFT). It takes the broadband ADC stream and simultaneously demodulates  $N$  equally-spaced frequency channels, analogous to a digital spectrum analyser. Advantages over the single-tone DDS approach include:

- No need to retune the ADC NCO between qubits.
- Computational cost scales as  $O(N \log N)$  rather than  $O(N)$  independent demodulators.
- Sidesteps the ADC-frequency-sweep limitation: each PFB output channel is a fixed digital filter, not a tunable DDS.

**[TODO: Describe PFB implementation status and preliminary results, if available.]**

## 6 Real-Time Feedback and Mid-Circuit Measurement

A key motivation for FPGA-based control is the ability to execute *real-time* conditional logic: perform a measurement, make a decision, and apply a corrective pulse — all on the FPGA, without a round-trip to the host CPU. This section describes the tProcessor-64 instruction set, demonstrates the conditional branching primitive, and outlines the active qubit reset protocol.

### 6.1 tProcessor Architecture

The tProcessor-64 is a soft-processor synthesised in the FPGA fabric. It executes a custom instruction set tailored to pulse generation and readout sequencing. Key features include:

- **Register file:** 32 general-purpose 64-bit registers per channel page, plus cross-page registers for inter-channel synchronisation.
- **Pulse triggers:** dedicated instructions that fire DAC signal generators or arm ADC acquisition windows.
- **Timing instructions:** `synci` (advance tProcessor time without touching channel timelines), `sync_all` (synchronise all channels), `wait_all` (stall until all readout windows close).
- **Conditional branch:** `condj page, reg_a, op, reg_b, label` jumps to `label` if `reg_a op reg_b` is true.

All branching targets are resolved at *compile time* by the Python assembler: Python's sequential execution of `body()` builds an instruction list, `label()` records the current position as a named address, and the assembler backpatches jump targets before loading the program onto the tProcessor. By the time the program runs, only integer addresses exist; there is no runtime symbol table.

## 6.2 Conditional Logic Demonstration

The QICK Demo 3 notebook demonstrates the core conditional-branch primitive:

```

1 def body(self):
2     cfg = self.cfg
3     self.trigger(adcs=[cfg["ro_ch"]],
4                 adc_trig_offset=cfg["adc_trig_offset"])
5     # condj: jump to SKIP_PULSE if number < threshold
6     self.condj(0, 2, '<', 1, 'SKIP_PULSE')
7     self.pulse(ch=cfg["res_ch"])          # skipped if condition true
8     self.label('SKIP_PULSE')
9     self.wait_all()
10    self.sync_all(self.us2cycles(cfg["relax_delay"]))

```

Registers 1 and 2 are pre-loaded with the threshold and the test value via `regwi`. When `number=100`, `threshold=50`: the condition `100 < 50` is false, the pulse fires, and the ADC captures a loopback signal. When `number=10`, `threshold=50`: the condition is true, the pulse is skipped, and the ADC trace is flat noise.

**Timing subtlety.** Inside a `body()` with an open ADC acquisition window, `synci` must be used to advance time between pulses, not `sync_all`. `sync_all` advances all channel timelines to the furthest point, which is the end of the ADC window; subsequent pulses would then be scheduled after the window has closed. `synci` advances only the tProcessor's internal counter, leaving the ADC timeline undisturbed.

## 6.3 $\pi$ Pulse Calibration

A  $\pi$  pulse rotates the qubit state by  $180^\circ$  on the Bloch sphere:  $|0\rangle \rightarrow |1\rangle$  and  $|1\rangle \rightarrow |0\rangle$ . It is a fundamental primitive for qubit initialisation (active reset) and single-qubit gates.

The  $\pi$  pulse amplitude is calibrated with a *Rabi experiment*: the qubit drive pulse amplitude is swept from zero, and the readout signal oscillates between the  $|0\rangle$  and  $|1\rangle$  IQ clusters as the qubit is driven through successive half-turns on the Bloch sphere. The first amplitude at which the signal crosses from the  $|0\rangle$  cluster to the  $|1\rangle$  cluster corresponds to the  $\pi$  pulse. The Rabi sequence is:

1. Prepare qubit in  $|0\rangle$  (wait  $\gg T_1$ ).
2. Apply qubit drive pulse at frequency  $\omega_{01}$  with variable amplitude  $A$  and fixed duration.
3. Read out via the resonator.

Note that the qubit drive tone is at  $\omega_{01}$  (the qubit transition frequency, typically 5 GHz to 7 GHz for fluxonium), which is distinct from the resonator frequency  $\omega_r$  used for readout. Two separate generator channels are therefore required: one for readout, one for qubit control.

#### 6.4 Active Qubit Reset

Active reset is the process of rapidly returning the qubit to  $|0\rangle$  by measuring its state and applying a corrective  $\pi$  pulse only if the qubit is found in  $|1\rangle$ :

1. **Dispersive readout**: fire the readout tone and acquire the accumulated IQ result.
2. **Thresholding**: the readout firmware compares the IQ result against a pre-calibrated threshold and writes a binary decision (0 or 1) into a tProcessor register.
3. **Conditional  $\pi$  pulse**: `condj` checks the decision register. If the qubit is in  $|1\rangle$ , a calibrated  $\pi$  pulse is applied on the qubit drive channel. Otherwise, the pulse is skipped.
4. **Wait**: allow any residual photons in the resonator to decay before starting the experiment.

Because the entire decision-to-pulse path runs inside the tProcessor, the latency between completing the readout window and firing the corrective pulse is determined solely by FPGA clock cycles. For the ZCU216 operating at a 384 MHz fabric clock, this latency is of order tens of nanoseconds — orders of magnitude faster than a software round-trip.

#### 6.5 Latency Budget

Table 3 gives an estimated latency breakdown for the active reset sequence.

Table 3: Estimated latency contributions in the active reset loop.

| Stage                     | Latency                  | Notes               |
|---------------------------|--------------------------|---------------------|
| Readout pulse duration    | 0.5 $\mu$ s to 2 $\mu$ s | Depends on $\kappa$ |
| ADC integration window    | 0.5 $\mu$ s to 2 $\mu$ s |                     |
| FPGA decision (threshold) | $\sim 10$ ns             | tProcessor cycles   |
| Conditional branch        | $\sim 3$ ns              | 1 tProc cycle       |
| $\pi$ pulse duration      | 50 ns to 500 ns          | Depends on $T_1$    |
| Total (excl. readout)     | $< 1$ $\mu$ s            |                     |

The dominant latency is the readout integration window; the digital decision and conditional fire together contribute less than 100 ns, well within qubit coherence times of 10  $\mu$ s to 100  $\mu$ s for state-of-the-art fluxonium devices.



## 7 ML-Assisted State Discrimination

### 7.1 IQ Data and the Discrimination Problem

Each single-shot readout produces an averaged  $(I, Q)$  pair — a point in the complex plane. Repeated preparation and measurement of the qubit in the same state produces a cloud (cluster) of such points, whose centroid and spread depend on the readout power, resonator parameters, and integration time. Two such clusters, one for  $|0\rangle$  and one for  $|1\rangle$ , must be distinguished reliably.

For ideal Gaussian clusters of equal covariance, the optimal linear classifier is the perpendicular bisector of the line segment joining the two centroids:

$$d(\mathbf{x}) = (\boldsymbol{\mu}_1 - \boldsymbol{\mu}_0)^\top \boldsymbol{\Sigma}^{-1} \left( \mathbf{x} - \frac{1}{2}(\boldsymbol{\mu}_0 + \boldsymbol{\mu}_1) \right) \underset{\hat{y}=0}{\overset{\hat{y}=1}{\geq}} 0, \quad (9)$$

where  $\boldsymbol{\mu}_0, \boldsymbol{\mu}_1 \in \mathbb{R}^2$  are the class centroids and  $\boldsymbol{\Sigma}$  is the common covariance matrix (Linear Discriminant Analysis, LDA). When the covariances differ, Quadratic Discriminant Analysis (QDA) uses the class-specific covariances and yields a curved decision boundary.

In practice, residual state preparation errors, qubit  $T_1$  decay during the readout pulse, and higher-level leakage ( $|2\rangle, |3\rangle$ ) distort the clusters from ideal Gaussians. These effects motivate the use of more expressive classifiers.

### 7.2 Lightweight Classifiers for FPGA Deployment

For on-FPGA deployment, the classifier must be computable using only fixed-point arithmetic and a small number of multiply-accumulate operations. The key constraints are:

- **Latency:** the decision must complete within one tProcessor cycle budget ( $\lesssim 10$  ns) to avoid extending the readout dead time.
- **Resource usage:** the classifier must fit in the available DSP slices and block RAM of the FPGA without displacing the readout and tProcessor logic.
- **Trainability:** the classifier must be trainable on a laptop from a few thousand calibration shots.

The following classifiers are evaluated in this work:

**Linear threshold (baseline).** A single fixed-point multiply-add and a comparison. Equivalent to LDA on the 1D projection of the IQ data onto the inter-centroid axis. Minimal resource usage, but suboptimal when cluster covariances differ significantly.

**Quadratic threshold.** A quadratic function of  $(I, Q)$  derived from QDA. Requires evaluating a  $2 \times 2$  matrix product, achievable with 4 multiply-accumulates. Handles asymmetric clusters.

**Small neural network.** A two-layer perceptron with inputs  $(I, Q)$ , a hidden layer of 8–16 neurons with ReLU activations, and a single sigmoid output. The hidden layer can be computed as a matrix-vector product; ReLU maps to a conditional copy. This architecture fits in  $\lesssim 200$  18-bit DSP slices on the ZCU216 FPGA fabric.

### 7.3 Training and Evaluation Pipeline

The training pipeline operates as follows:

1. **Calibration data collection:** prepare the qubit in  $|0\rangle$  (passive thermalisation or active reset) and  $|1\rangle$  ( $\pi$  pulse followed by reset) and collect  $N_{\text{cal}}$  IQ shots per class using the accumulated readout mode.
2. **Preprocessing:** subtract the per-class mean (centring) and optionally rotate the IQ frame so that the inter-centroid axis aligns with the  $I$  axis.
3. **Training:** fit the chosen classifier using scikit-learn (LDA/QDA) or PyTorch (neural network) on the host laptop.
4. **Quantisation:** convert the trained weights to fixed-point representation compatible with the FPGA DSP pipeline.
5. **Deployment:** write the quantised weights into the tProcessor data memory or a dedicated classification IP block.

### 7.4 Fidelity Metrics

Table 4 [TODO: fill with actual measured values] summarises the expected readout fidelity and latency for each classifier.

Table 4: Comparison of state discrimination classifiers. All values are estimated / to be measured.

| Classifier       | $\mathcal{F}$ (%) | FPGA latency (ns) | DSP slices |
|------------------|-------------------|-------------------|------------|
| Linear threshold | [TODO: –]         | <5                | 2          |
| QDA              | [TODO: –]         | <10               | 8          |
| MLP (8 neurons)  | [TODO: –]         | <20               | ~50        |

[TODO: Add IQ scatter plots (one per classifier) showing the decision boundary overlaid on calibration data, and the ROC curve comparison.]

## 8 Conclusions and Outlook

### 8.1 Summary

This thesis has presented a general-purpose readout architecture for superconducting qubits, built on the QICK framework and the Xilinx ZCU216 RFSoc platform at Qilimanjaro Quantum Tech.

The main contributions are:

1. **System integration:** The ZCU216 board, XM655 analogue front-end, and QICK firmware were brought up from scratch, including SSH access, PYNQ configuration, and a persistent Pyro4 remote-operation server that preserves calibration state across notebook sessions.
2. **Phase-coherent readout calibration:** A systematic calibration procedure was developed that measures the random DAC–ADC DDS phase offset at board power-on. The linear phase-vs-frequency model  $\phi(f) = \phi_0 - 360^\circ \tau f$  was validated, and the fitted `res_phase` value was incorporated into the standard experiment configuration.

3. **Readout data-path:** The trade-offs between decimated (waveform) and accumulated (averaged IQ) readout modes were characterised. The  $1/\sqrt{N_{\text{reps}}}$  noise scaling of accumulated readout was confirmed, reducing single-shot noise from  $\sim 1.9$  to  $\sim 0.14$  ADU at 1000 repetitions. The resonator spectroscopy workflow was established for both single-qubit and multiplexed scenarios.
4. **Real-time feedback:** The tProcessor-64 conditional branch (`condj`) was demonstrated as the core primitive for mid-circuit measurement. An active qubit reset sequence was designed, combining dispersive readout, thresholding, and a calibrated  $\pi$  pulse, with a digital decision-to-fire latency below 100 ns.
5. **ML-assisted discrimination:** Linear Discriminant Analysis, QDA, and a small neural network were evaluated as state classifiers. **[TODO: Summarise actual experimental findings once measurements are complete.]**

## 8.2 Outlook

Several natural extensions of this work are planned:

**Multi-qubit readout.** The current architecture supports a single readout tone. Extending to multiplexed readout via the QICK PFB firmware would allow simultaneous readout of all qubits in Qilimanjaro’s multi-qubit chips, which is a prerequisite for running quantum error-correction cycles.

**On-FPGA classifier deployment.** The machine-learning classifiers developed in Chapter 7 are currently evaluated in software. Their deployment as fixed-point FPGA IP blocks — so that the tProcessor receives a binary decision within a single clock cycle — is the next step toward closing the active-reset feedback loop at full speed.

**Integration with Qilimanjaro’s software stack.** The QICK-based readout layer developed here should be integrated with Qilimanjaro’s higher-level quantum programming interfaces (circuit compilation, calibration databases, experiment management) to provide a production-ready control stack.

**Fluxonium-specific readout optimisation.** The dispersive shift landscape for fluxonium qubits is two-dimensional in the  $(\Phi_z, \Phi_x)$  flux space. An ML-assisted optimisation of the readout circuit parameters — coupling capacitance, resonator frequency — to maximise the dimensionless cavity pull over a large flux region is a complementary research direction being pursued in a related BSc project within the team.

## Bibliography

- [1] Leandro Stefanazzi et al. QICK: Quantum instrumentation control kit for superconducting quantum computers. *Review of Scientific Instruments*, 93(4):044709, 2022. arXiv:2110.00557.
- [2] Philip Krantz, Morten Kjaergaard, Fei Yan, Terry P. Orlando, Simon Gustavsson, and William D. Oliver. A quantum engineer's guide to superconducting qubits. *Applied Physics Reviews*, 6(2):021318, 2019.
- [3] Jens Koch, Terri M. Yu, Jay Gambetta, A. A. Houck, D. I. Schuster, J. Majer, Alexandre Blais, M. H. Devoret, S. M. Girvin, and R. J. Schoelkopf. Charge-insensitive qubit design derived from the Cooper pair box. *Physical Review A*, 76(4):042319, 2007.
- [4] Vladimir E. Manucharyan, Jens Koch, Leonid I. Glazman, and Michel H. Devoret. Fluxonium: Single Cooper-pair circuit free of charge offsets. *Science*, 326(5949):113–116, 2009.
- [5] Alexandre Blais, Arne L. Grimsmo, S. M. Girvin, and Andreas Wallraff. Circuit quantum electrodynamics. *Reviews of Modern Physics*, 93(2):025005, 2021. arXiv:2005.12667.
- [6] Alexandre Blais, Ren-Shou Huang, Andreas Wallraff, S. M. Girvin, and R. J. Schoelkopf. Cavity quantum electrodynamics for superconducting electrical circuits: An architecture for quantum computation. *Physical Review A*, 69(6):062320, 2004. arXiv:cond-mat/0402216.
- [7] Jay Gambetta, Alexandre Blais, Maxime Boissonneault, Andrew A. Houck, D. I. Schuster, and S. M. Girvin. Protocols for optimal readout of qubits using a continuous quantum nondemolition measurement. *Physical Review A*, 76(1):012325, 2007.

## A Key Code Listings

This appendix collects the most important Python/QICK code snippets used in this thesis. Full notebooks are available in the accompanying GitHub repository.

### A.1 Phase Calibration Programme

```

1  class PhaseCalProgram(AveragerProgram):
2      """Sweeps frequency and measures DAC-ADC phase offset."""
3
4      def initialize(self):
5          cfg = self.cfg
6          self.declare_gen(ch=cfg["res_ch"], nqz=1)
7          self.declare_readout(ch=cfg["ro_ch"],
8                               length=cfg["readout_length"],
9                               freq=cfg["pulse_freq"],
10                              gen_ch=cfg["res_ch"])
11          freq_reg = self.freq2reg(cfg["pulse_freq"],
12                                   gen_ch=cfg["res_ch"],
13                                   ro_ch=cfg["ro_ch"])
14          self.set_pulse_registers(ch=cfg["res_ch"],
15                                   style="const",
16                                   freq=freq_reg,
17                                   phase=0,
18                                   gain=cfg["pulse_gain"],
19                                   length=cfg["length"])
20
21          self.synci(200)
22
23      def body(self):
24          self.measure(pulse_ch=self.cfg["res_ch"],
25                      adcs=[self.cfg["ro_ch"]],
26                      adc_trig_offset=self.cfg["adc_trig_offset"],
27                      wait=True,
28                      syncdelay=self.us2cycles(self.cfg["relax_delay"]))

```

### A.2 Fast Amplitude Sweep with RAveragerProgram

```

1  class SweepProgram(RAveragerProgram):
2      """Sweeps DAC gain inside the FPGA without Python overhead."""
3
4      def initialize(self):
5          cfg = self.cfg
6          self.declare_gen(ch=cfg["res_ch"], nqz=1)
7          self.declare_readout(ch=cfg["ro_ch"],
8                               length=cfg["readout_length"],
9                               freq=cfg["pulse_freq"],
10                              gen_ch=cfg["res_ch"])
11          freq_reg = self.freq2reg(cfg["pulse_freq"],
12                                   gen_ch=cfg["res_ch"],
13                                   ro_ch=cfg["ro_ch"])
14          self.set_pulse_registers(ch=cfg["res_ch"],
15                                   style="const",
16                                   freq=freq_reg,
17                                   phase=cfg["res_phase"],
18                                   gain=cfg["start"],
19                                   length=cfg["length"])

```

```

20     self.r_rp    = self.ch_page(cfg["res_ch"])
21     self.r_gain = self.sreg(cfg["res_ch"], "gain")
22     self.synci(200)
23
24     def body(self):
25         self.measure(pulse_ch=self.cfg["res_ch"],
26                     adcs=[self.cfg["ro_ch"]],
27                     adc_trig_offset=self.cfg["adc_trig_offset"],
28                     wait=True,
29                     syncdelay=self.us2cycles(self.cfg["relax_delay"]))
30
31     def update(self):
32         self.mathi(self.r_rp, self.r_gain, self.r_gain,
33                   '+', self.cfg["step"])

```

### A.3 Active Reset Sequence (Pseudocode)

---

**Algorithm 1** Active qubit reset

---

**Require:** Calibrated  $\pi$ -pulse amplitude  $A_\pi$  and readout threshold  $\theta$

Trigger ADC readout window

Play readout tone on resonator channel

**wait** for readout window to close

$b \leftarrow \text{FPGA\_threshold}(I + jQ, \theta)$

$\triangleright b = 1$  if qubit in  $|1\rangle$

**regwi**  $r_b \leftarrow b$

**condj**  $r_b < 1$  **goto** SKIP\_PI

Play  $\pi$  pulse on qubit drive channel

**label** SKIP\_PI

**synci**  $t_{\text{relax}}$

$\triangleright$  Allow cavity photons to decay

---